

AI 原生鸿蒙软件工程实践

个性化需求 · 生成式交互 · 智能体架构 · 数据飞轮

2026 秋季 | 18 周 × 3 小时 | AI Native

软件不再只为“平均用户”设计

同一个“取快递”目标

行动不便的学生需要长期可信帮助；正在上课的学生需要静默通知；新生需要先理解路线；留学生需要跨语言交流。

每个人都有特殊情境

身份、时间、地点、关系、能力、隐私、情绪和风险偏好共同决定这一次任务真正需要什么。

需求不可能全部预先枚举

固定页面和统一流程只能覆盖常见情况，难以适应不断出现的个人微需求。

AI 让临时定制成为可能

AI 可以理解目标、追问约束、组合工具，并为本次任务即时生成合适的交互。

这门课研究：如何建立一个稳定、可信的软件底座，为每个人动态实现不同的需求和体验。

对话是入口，卡片是行动界面

用户直接表达目标

“我晚上九点下课，想找一个从北门经过 5 号宿舍楼的人帮我取快递，最好是以前合作过的同学。”

AI 理解与澄清

识别地点、时间、熟人偏好和缺失信息

可信工具执行

匹配、发布、通知、订单与评价都通过受控接口完成

界面随任务即时出现

确认卡

展示 AI 理解的地点、时间、报酬和约束

方案比较卡

比较立即匹配、稍后匹配或熟人优先

进度与风险卡

展示任务状态、下一步、撤销与安全确认

自然语言负责表达意图，生成式 UI 负责结构化确认，可信工具负责真实业务操作。

AI Native：贯穿整个学期的课程项目



从上学期的 CampusHub 1.0 出发，将固定功能平台演化为面向每个人的校园智能体。

不是重新做一遍 CRUD，而是完成一次 AI 原生软件演化

完成课程后，你将形成六类能力

个性化需求工程

区分稳定需求、长期偏好和一次性情境化微需求。

Human-AI Interaction

设计对话、卡片、确认、解释、撤销、降级和人工接管。

鸿蒙应用开发

用 ArkTS、ArkUI 和 Stage 构建对话与生成式 UI 客户端。

现代语言与并发

比较 ArkTS 与仓颉，并实现安全的工具或事件服务。

AI 原生架构

设计 Agent、模型网关、工具网关、LLM 部署与可靠性。

数据治理与飞轮

让本体、事件、质量、评测和实验持续促进业务。

Part 1

从特殊需求到生成式交互

第 1—3 周：先理解每个人的不同

W1 AI 原生软件工程

接管 CampusHub 1.0，识别稳定底座、动态体验、技术债和数据缺口。

W2 个性化需求与业务本体

建立用户、目标、资源、约束、任务、匹配、信用和授权等领域概念。

W3 对话与抛弃式 UI

设计澄清、确认、修改、撤销、卡片、固定页面和人工接管。

第一次评审要证明

同一目标，五种特殊需求

不能只给所有用户设计同一种表单和流程

业务本体能够约束 AI

概念、关系、规则和允许的操作必须清晰

交互能够澄清而不是猜测

AI 对不确定信息要追问，并允许用户纠正

第 4—7 周：在鸿蒙端实现动态体验

W4 ArkTS 领域模型

类型、接口、异步、状态机、业务不变量和自动化测试。

W5 ArkUI Schema Renderer

声明式 UI、组件白名单、Schema 校验、无障碍和多设备适配。

W6 流式对话协议

SSE/WebSocket、文本片段、UI Schema、工具进度、断线与恢复。

W7 可信工具与降级

权限、确认、幂等、补偿、固定页面和人工服务回退。

生成 UI 的安全边界

模型只生成声明式 Schema

不直接在线执行模型生成的 ArkTS、JavaScript 或 SQL

客户端只渲染白名单组件

验证版本、字段类型、权限和 Action 范围

副作用操作必须确认

发布、接单、取消等由可信工具执行并留下审计记录

一条流式消息可以同时驱动对话、界面和任务

assistant.delta

逐步显示
自然语言

ui.schema

生成卡片
或表单

tool.pending

等待确认
显示进度

tool.result

返回业务
执行结果

task.state

更新任务
状态

自然语言不是控制协议

客户端不能解析模型说了什么来决定是否发布订单，而应依据明确的消息类型、Schema 和工具状态。

任务状态不能只放在对话上下文

业务状态由系统保存和校验，模型只能读取允许的状态并提出下一步建议。

Part 2

语言、智能体与 LLM 系统架构

第 8—9 周：用两种语言理解类型与并发

学习内容	ArkTS	仓颉	项目应用
领域建模	接口、类、空安全	值/引用类型、模式匹配	用户目标、任务状态、UI Schema
并发机制	TaskPool、Worker、Sendable	语言线程与并发安全	事件处理、工具服务、任务执行
错误表达	类型与异常	类型、模式和错误处理	明确失败、重试与降级语义
课程角色	完成鸿蒙客户端	实现小型后端组件	形成有证据的语言选型 ADR

第 8 周：同一模型双语言实现

比较类型约束、可读性、错误处理和工具链，而不是只背语法差异。

第 9 周：仓颉工具或事件服务

实现可被 AI Native 调用的并发组件，并用测试或性能数据解释其价值。

第 10 周：智能体不是一个无限权限的聊天模型



第 11 周：LLM 部署必须同时考虑能力、成本和数据

云端模型

能力强、部署快；需要管理调用成本、网络依赖和数据出域风险。

校内或私有模型

数据可控、可定制；需要承担算力、部署、升级和运维成本。

端云混合

敏感和低延迟任务留端，复杂推理上云；模型路由与版本管理更复杂。

Model Gateway 管理

- 模型路由与版本
- 配额、缓存与成本
- 超时、重试与熔断
- Prompt 和 Context 版本

团队必须回答

- 哪些数据可以发送到哪里？
- 一次任务消耗多少 Token 和费用？
- 模型不可用时如何继续服务？
- 为什么为这个任务选择这个模型？

第 12 周：AI 系统也要扛得住、救得回

高并发与容量

流式会话

长连接数量、持续时间、心跳和断线恢复

模型算力

并发请求、Token/s、上下文长度、排队和背压

成本容量

单次成本、用户配额、峰值预算和缓存收益

高可用与恢复

模型降级

切换模型、简化任务或退回规则和固定页面

工具可靠性

幂等、超时、重试、补偿和最终一致性

端到端观测

贯通客户端、Agent、模型和业务工具的 trace

M3 不是提交一张架构图，而是用压测、SLO 和故障注入证明系统能够运行。

模型超时、工具重复调用和流式连接中断都要实际演练

Part 3

让业务知识与数据持续增值

第 13—14 周：先让数据具有业务语义和责任边界

AI 交互全过程事件

- 目标表达、意图假设、澄清和用户纠正
- Schema 生成、拒绝、渲染和卡片操作
- 工具请求、用户确认、成功、失败和补偿
- 任务完成、放弃、降级和人工接管

同时记录模型、本体、Prompt、Schema 和工具版本。

个人记忆与数据治理

声明偏好

用户明确表达，可以查看、修改和撤回

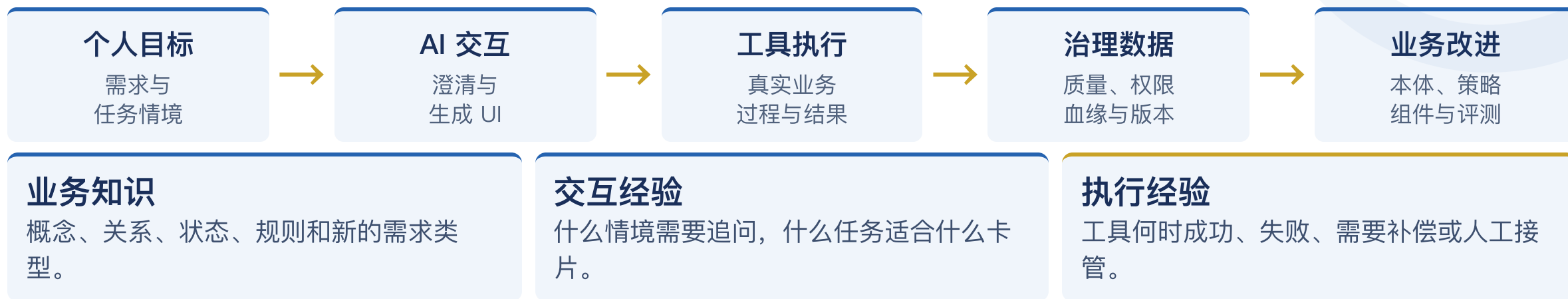
推测偏好

必须标明推测，不得直接固化为人格标签

临时上下文

仅服务本次任务，到期后删除或匿名化

数据飞轮积累的不只是点击记录



数据飞轮不是“收集越多越好”，而是让被授权、可解释、可追溯的数据持续改善业务。

每一次服务既完成一个任务，也为下一次更好的服务积累知识

第 15—16 周：用评测和实验证明系统真的变好

固定评测集

- 正常、模糊和信息缺失需求
- 不同个人偏好和特殊情境
- 恶意输入、越权和隐私攻击
- 错误 Schema、工具失败和模型超时

评测意图、澄清、Schema、工具选择、安全、延迟和成本。

数据飞轮实验

建立基线

例如所有任务只使用纯文本对话

实施干预

为时间、地点和比较场景动态生成卡片

验证结果

比较完成时间、修改率、误操作和满意度

第 17 周：主动寻找 AI 系统最危险的失败方式

提示注入与越权

诱导模型忽略规则、查询他人数据或调用未授权工具。

恶意生成式 UI

伪造按钮含义、绑定危险 Action、请求过度权限或制造误导确认。

业务规则绕过

重复创建订单、非法状态迁移、未经确认执行和信用误判。

模型与网络故障

超时、输出中断、重复消息、模型切换和任务状态丢失。

隐私泄露

在推荐理由、日志、Prompt 或评测数据中暴露身份与行为轨迹。

修复并回归

记录攻击、结果、风险、修复和复测，形成永久安全测试。

Part 4

项目节奏、考核与学习方式

四次里程碑逐步形成完整系统



现场运行、解释设计、处理变化，比预先录制的完美演示更重要

成绩来自工程判断、验证和业务结果

考核项目	比例	主要内容
华为鸿蒙开发初级证书	30%	完成认证学习与考试，达到鸿蒙应用开发基础能力要求
M1 个性化需求与 HAI	15%	特殊需求、业务本体、个人上下文、对话与卡片
M2 鸿蒙与生成式 UI	15%	客户端、Renderer、流式协议、工具确认与降级
M3 智能体与可靠性	20%	Agent、模型部署、成本、容量、SLO 与故障恢复
M4 数据治理、评测与飞轮	15%	事件、质量、血缘、权限、评测实验、现场挑战与发布质量
个人答辩	5%	个人贡献、架构解释、现场诊断和 AI 责任

AI 可以广泛参与开发，但不能替你承担责任

可以委派给 AI

- 需求样本与边界场景生成
- 本体候选与规则检查
- ArkTS、仓颉和后端代码
- UI Schema、测试与攻击样本
- 架构比较与代码评审

团队必须亲自负责

- 稳定架构和安全边界
- 业务本体和工具契约
- 生成结果的测试与验证
- 隐私、成本和失败恢复
- 最终业务结果与责任

AI 生成只是候选方案；经过验证、能够解释并愿意负责，才是软件工程。

每周课堂是一个 AI 原生工程工作室

50 min

概念、架构与案例

40 min

教师与 AI 示范

70 min

团队开发与实验

20 min

随机评审与反馈

课堂讲稳定的工程原则

状态、边界、并发、一致性、故障、验证、安全、隐私和数据责任。

平台 API 通过实验掌握

使用官方文档、示例和 AI 快速学习变化中的鸿蒙与模型接口。

失败案例进入知识库

保留错误需求、失败 Schema、工具异常和模型误导，成为后续评测资产。

项目持续演化

每次评审产生本体、协议、组件、规则、测试或数据资产的新版本。

第 18 周：现场完成一个从未预设的特殊需求

“我脚受伤了，今晚需要取三个快递，但不希望把宿舍号直接告诉陌生人；我可以请对方喝咖啡，而且希望以后还能联系同一个人。”

理解与澄清

识别个人目标、隐私约束、缺失信息和长期偏好候选。

生成与确认

现场生成本次任务需要的卡片，并允许用户修改、撤销和确认。

工具与状态

调用匹配、隐私位置和订单工具，正确更新真实业务状态。

失败与降级

模型或工具失败后，任务仍可恢复、回退或转人工处理。

数据与飞轮

说明哪些信息保留、哪些被抛弃，以及本次经验如何改善未来服务。

个人答辩

解释代码、架构、数据和 AI 决策，现场定位或修改问题。

兴趣小组：以真实外部成果替代统一大作业

源码阅读与社区贡献

阅读 OpenHarmony 或鸿蒙生态源码，提交 Issue、文档、补丁或 Pull Request；也可开展论坛发帖、教程案例分享、鸿蒙布道师等活动。

鸿蒙相关竞赛

参加华为、OpenHarmony 生态或合作单位认可的竞赛，提交可运行作品并取得正式参赛或评审成果。

鸿蒙项目

开展 Skill 移植、桌面鸿蒙开发或其他生态项目，形成可发布、可合并、可部署或被实际采用的成果。

替代规则

- 加入兴趣小组后，不参加课程统一大作业。
- 兴趣小组成果替代大作业部分的课程评价。
- 仅有参与过程、不形成外部成果，不能作为完成证据。

成果认定

- 获得华为的正式认可或采用。
- 获得 OpenHarmony 相关企业的认可或采用。
- 获得开源社区的合并、发布、收录、推荐、活动认定或正式反馈。

华为鸿蒙开发初级认证仍须参加。兴趣小组只替代统一大作业，不替代 30% 的认证要求。

为每个人即时创造可信的软件体验

理解特殊需求 · 生成临时交互 · 调用可信工具 · 积累业务知识 · 持续改善下一次服务